

WebDriverIO

Create a project:

```
npm init wdio your-project-file-name
```

Then select the framework *mocha*

Run the project:

```
npm run xxxx
```

Some minor Improvement:

1. In "wdio.conf.js" file change this line into this:

```
loglevel: 'error'
```

2. Maximize the window:

In "wdio.conf.js" file change this line into this:

```
before: function (capabilities, specs) {
```

```
  browser.maximizeWindow()
```

```
}
```

3. Suggestions Library Setup: Create file named "jsconfig.json" & paste the code in it.

```
{
  "compilerOptions": {
    "types": [
      "node",
      "@wdio/globals/types",
      "@wdio/mocha-framework"
    ]
  }
}
```

```
}  
}
```

Element CSS Selector:

if dropdown or search option is working in Realtime then delete these values in the event listeners:

*****For those who cannot find 'blur' event, alternatively you can remove ones in 'focusout'**

```
//button/i[contains(text(), "ABC")]
```

Another way to get text:

```
await $('button = Add Text')
```

Right button click:

```
await variable.click({ button: 'right' })
```

`variable.scrollToView()` means it will scroll to that element which is stored at variable.

`setValue("String")` - insert the string in the input

`addValue("String")` - means it will add the value with the given input previous text
..Like some phone number field has country code.

`await browser.keys(["Control","a"]);` — it means it will select all the text. when performing multiple keyboard action then it is written like this.

Tab switching:

```
const totaltab = await browser.getWindowHandles();//getting the open tabs info
await browser.switchToWindow(totaltab[1]); //switch to opened new tab
await browser.pause(3000);
await browser.switchToWindow(totaltab[0]); //switch to previous tab
await browser.pause(3000);
```

Handle alert:

await browser.acceptAlert(); //for accept the alert.

await browser.dismissAlert(); //for not accepting or dismiss the alert

await browser.sendAlertText("WebdriverIO is fun"); // for sending the text in input box of alert.

isAlertOpen() //return the boolean value

getAlertText() // returns the alert text

XPath:

Xpath Methods

1. We can catch everything that we caught using CSS selector (id, class, attribute, text)
2. Contains() method
 - a. `//button[contains(@type,'sub')]`
3. And / Or in xpath
 - a. `//*[@type='submit' or @name='btnReset']`
4. Starts with
 - a. `//a[starts-with(@href,'http://www.cefalo.com')]`
 - b. Logbt andvjhad
 - c. `//button[starts-with(@id,'logbt')]`
5. Text() method
 - a. `//td[text()='Delete']`
6. Following
 - a. `//button[@type='sub']/following::input`
7. Child
 - a. `//form[@id='java_technologies']/child::li`
8. Following-sibling
 - a. `//div[@type='submit']/following-sibling::input`
9. Parent
 - a. `//div[@id='someid']/parent::div`

`dragelementvariable.dragAndDrop(dropelementVariable);` ///for drag and drop an element but these value stored in function...so need to store in a variable.

```
async dragTheButtonAndDropInsideTheDiv() {  
  const dragElement = await this.theButtonIwantToDrag  
  const dropTarget = await this.thePlaceIwantToDrop  
  dragElement.scrollIntoView()  
  await browser.pause(3000)  
  dragElement.dragAndDrop(dropTarget)  
}
```

`elementvariable.moveTo()` //for mouse hover

POM Structure:

test/pageobjects Folder:

--class creation:

login_page.js

```

import { $ } from '@wdio/globals';
import Page from './page.js';

class loginpage{
  //elems gathering
  get Usernamefield(){
    return $("#username");
  }

  get passwordfield(){
    return $("field[id='pass']");
  }

  get loginbutton(){
    return $("#login");
  }

  //minor action
  async typeonusernamefield(username){
    (await this.Usernamefield).setValue(username);
    await browser.pause(3000);
  }

  async typeonpasswordfield(password){
    (await this.passwordfield).setValue(password);
    await browser.pause(3000);
  }

  async clickonloginbutton(){
    (await this.loginbutton).click();
    await browser.pause(3000);
  }

  async gotologinpage(){
    await browser.url("xy.com");
  }
}

```

```

//major action of the page
async automatePage(username,password){
  await this.typeonusernamefield(username);
  await this.typeonpasswordfield(password);
  await this.clickonloginbutton();
}

}

export default new loginpage();

```

Now creating the test file:

test/specs folder:

loginpage_test.js

```

import loginpage from '../pageobjects/login_page.js';

describe("Login page Test", () => {

  it("The user should login with valid credentials", async() => {
    await loginpage.automatePage("admin","admin");
  })

})

```

```

async dragTheButtonAndDropInsideTheDiv() {
  const dragElement = await this.theButtonIwantToDrag
  const dropTarget = await this.thePlaceIwantToDrop
  dragElement.scrollIntoView()
  await browser.pause(3000)
  dragElement.dragAndDrop(dropTarget)
}

```

Assertion:

Browser assertion

1. toHaveUrl()
2. toHaveTitle()

```

await expect(browser).toHaveUrl("https://play1.automationcamp.ir/order_submit.html")
//await expect(browser).toHaveTitle('Loginff')

```

Element matcher

toBeDisplayed (Exist + Present)

isExist() //maybe it is present in dom, but showed later after an action.

toBePresent

toBeClickable

toBeFocused

toHaveAttribute('attname','attvalue')

expect(input).toHaveValue("Nameapnar")

toBeDisabled

toBeEnabled

toBeSelected

toBeChecked

toHaveText

toBeDisplayedInViewport
Error message

HOME WORK

toBeElementsArrayOfSize
toHaveSize

```
const triggerButton = await $('#visibility_trigger')
const clickMeButton = await $('#visibility_target')
await expect(clickMeButton).toBeDisplayed()
```

We can add custom message for failed case in assertion.

```
await expect(disableButton).toBeDisplayedInViewport({message : 'I did not scroll to that element that is why the button din not get in the view'})
```

Making a Suite: wdio.conf.js

After specs: block it should present there.

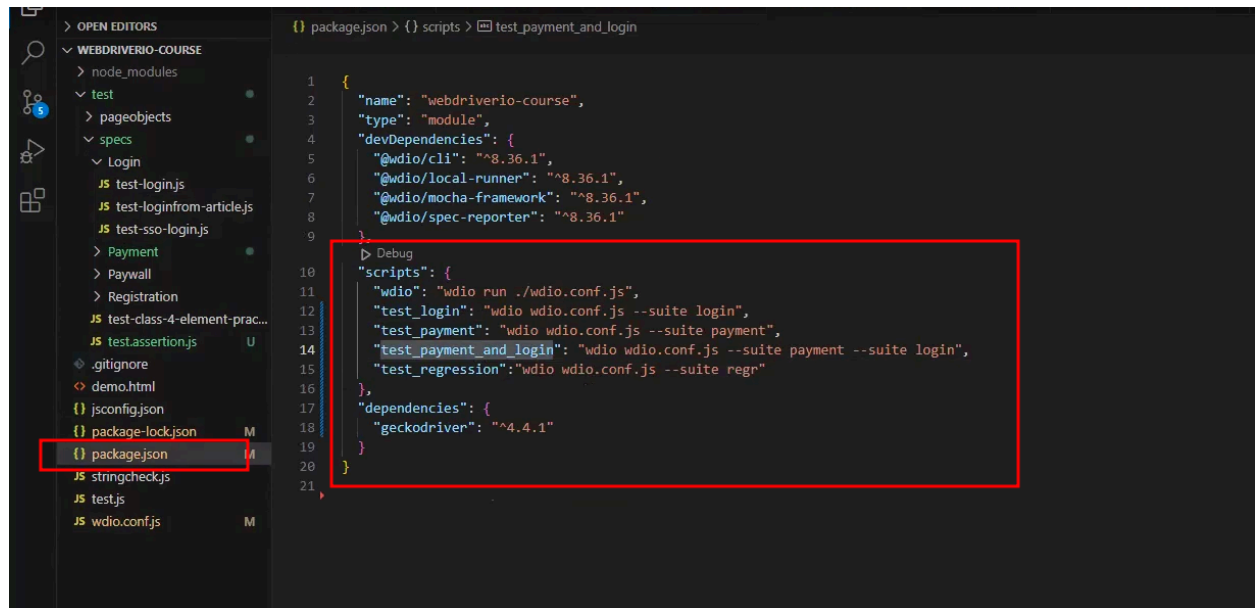
```
suite: {

  login: [
    './test/specs/**/*.test.e2e.js'
  ],

  registration: [
    './test/specs/**/*.test.e2e.js'
  ]

},
```

change according to the package.json file



Class-8-More Expect-Wait-Screenshot

How to set wait particular for a assertion:

```
await triggerbutton.click()
await expect(disableButton).toBeDisabled({wait:1000})
```

We can make after assertion message or take as screenshot in the custom function.

```

    await triggerbutton.click()
    function afterMyAssertion () {
        console.log("I am after Assertion ===== ")
    }
    await expect(disableButton).toBeEnabled({afterAssertion: afterMyAssertion})
}

```

Before assertion also same thing as after assertion.

```

it("Understanding afterAssertion", async () => {
    await browser.url('https://play1.automationcamp.ir/expected_conditions.html')
    await browser.pause(2000)
    let name = "Noman"
    let birthday = "2/2/2/"
    const disableButton = await $('#enabled_target')
    const triggerbutton = await $('#enabled_trigger')
    await disableButton.scrollIntoView()
    await browser.pause(2000)
    await triggerbutton.click()
    function afterMyAssertion () {
        console.log("I am after Assertion ===== ")
    }
    function beforeMyAssertion () {
        console.log("I am before Assertion ===== ",name,birthday)
    }
    await expect(disableButton).toBeEnabled({beforeAssertion:beforeMyAssertion,afterAssertion: afterMyAssertion})
})

```

Now to take screenshot:

Create a file in the test folder named utility.js'

```

class Utility {

    async getCurrentTimeInSeconds() {
        return Math.floor(new Date().getTime() / 1000);
    }

    async takeScreenShotAsProof(asserTionDetails) {
        console.log("===== Assertion details", asserTionDetails.result)
        let result = asserTionDetails.result.pass
    }
}

```

```

    if(result === false){
      let curretTime = Math.floor(Math.random() * 10)
      browser.saveScreenshot(`./screenshots/${curretTime}-bug-image.png`)
    }
    else
      console.log("Failed to take Screenshot")

  }

}

export default new Utility();

```

Now in the test.js file follow accordingly.

1. Import the utility class.

```
import Utility from '../utility.js'
```

2. Now call the function from the "await expect(disableButton).toBeEnabled()" assertion line

```

it("Understanding afterAssertion", async () => {
  await browser.url('https://play1.automationcamp.ir/expected_conditions.htm')
  await browser.pause(2000)
  const disableButton = await $('#enabled_target')
  const triggerbutton = await $('#enabled_trigger')
  await disableButton.scrollIntoView()
  await browser.pause(2000)
  await triggerbutton.click()
  await expect(disableButton).toBeEnabled({wait: 7000, afterAssertion: Utility

```

String Manipulation at the assertion:

1. String option
 - a. ignoreCase
 - b. trim
 - c. Containing
 - d. atStart
 - e. atEnd
2. Number option
 - a. eq
 - b. lte
 - c. Gte

ignoreCase: It will ignore the text as case sensitive.

```
it("Practicing String Option for Element Matcher Assertion", async () => {  
  await browser.url('https://letcode.in/test')  
  await browser.pause(5000)  
  const firstHeaderElement = await $('//header/p[1]')  
  await expect(firstHeaderElement).toHaveText('In',{ignoreCase:true})  
})
```

trim: It will trim the whitespace after or before the string.

```
it("Practicing String Option for Element Matcher Assertion", async () => {  
  await browser.url('https://letcode.in/test')  
  await browser.pause(5000)  
  const firstHeaderElement = await $('//header/p[1]')
```

```
await expect(firstHeaderElement).toHaveText('In',{ignoreCase:true, trim:true  
})
```

Class-15-Allure Report,

For windows, these is the instructor's slide copy:

Install

npm install @wdio/allure-reporter --save-dev

Configure in wdio.js

```
['allure', {  
  outputDir: 'allure-results',  
  disableWebdriverStepsReporting: true,  
  disableWebdriverScreenshotsReporting: true,  
}]
```

Let's understand the configuration

Install command line tool for allure

- a. npm i allure-commandline
- b. Type allure in cmd
- c. See any problem, then follow the following solution

Open powershell in admin mode

Run these following

- a. Get-ExecutionPolicy
- b. Set-ExecutionPolicy Unrestricted
- c. Get-ExecutionPolicy -List
- d. Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
- e. After work setback to Set-ExecutionPolicy Restricted

Allure install:

```
npm install @wdio/allure-reporter --save-dev
```

setup allure in the wdio.conf.js file.

search for reporters and after spec word give comma and press enter then paste these code:

```
['allure', {  
  outputDir: 'allure-results',  
  disableWebdriverStepsReporting: true,  
  disableWebdriverScreenshotsReporting: true,  
}]
```

Add the lines in the **package.json** file in the **scripts** object

```
"show-report": "allure generate --clean allure-results && allure open"
```

```
"scripts": {  
  "wdio": "wdio run ./wdio.conf.js",  
  "show-report": "allure generate --clean allure-results && allure open",  
  "single-report": "allure generate --clean --single-file allure-results"  
},
```

```
//Note: "single-report": "allure generate --clean --single-file allure-results"  
// is added for the single file report, use any of this when needed.
```

Now at the wdio.conf.js file import this at the 1st of the code:

```
import allure from '@wdio/allure-reporter';  
import fs from 'fs';  
import path from 'path';  
import CustomCommands from './test/commands.js';
```

Now at the wdio.conf.js file search "afterTest: function" and replace the function with this code:

```
afterTest: async function (test, context, { error, result, duration, passed, retries }) {
  if (error) {
    // Take screenshot
    const screenshot = await browser.takeScreenshot();
    // Attach screenshot to Allure report
    allure.addAttachment('Screenshot on Failure', Buffer.from(screenshot, 'base64'));
  }
},
```

Now at the wdio.conf.js file search "before: function (capabilities, specs)" and replace the function with this code:

```
before: function (capabilities, specs) {

  CustomCommands();
  browser.maximizeWindow();
  const allureReportPath = path.join(process.cwd(), 'allure-report');
  if (fs.existsSync(allureReportPath)) {
    fs.rmdirSync(allureReportPath, { recursive: true });
  } else {
    console.log('Folder allure report does not exist');
  }
  const allureResultPath = path.join(process.cwd(), 'allure-results');
  if (fs.existsSync(allureResultPath)) {
    fs.rmdirSync(allureResultPath, { recursive: true });
  } else {
    console.log('Folder allure result does not exist');
  }
},
```

Now in the test.js file follow accordingly.

1. Import the fs & allure class.

```
import allure from '@wdio/allure-reporter';  
import fs from 'fs';
```

N.B: In the website, there are many elements that are not implemented correctly so XPath might be very long so we can mark them in allure report & we can report those in Jira task.

```
it("It should failed", async () => {  
  allureReporter.addLabel('feature', 'Signup');  
  allureReporter.addTag('fixme');  
  await expect(2).toBeGreaterThan(5)  
})
```

Here,

```
allure.addTag("fixme");//here upper picture it was allureReporter because import
```

Class-10-Mock, Shadow Element, Browser Event

Browser Events:

```
// Browser Events =====  
it.only('Browser Events', async () => {  
  await browser.navigateTo('https://play1.automationcamp.ir/')  
})
```

```

    await browser.pause(3000)
    await browser.url('https://letcode.in/test');
    await browser.pause(3000)
    await browser.back()
    await browser.pause(3000)
    await browser.forward()
    await browser.pause(3000)
    await browser.refresh()
    await browser.pause(3000)
  })

```

Shadow Element:

```

// Shadow Elements =====
it('Shadow Elements', async () => {
  await browser.url('https://practice.expandtesting.com/shadowdom');
  await browser.pause(3000)
  const innerEl = await $('#shadow-host').shadow$('#my-btn');
  await innerEl.click()
  await browser.pause(3000)
})

it('Shadow Elements', async () => {
  await browser.url('http://uitestingplayground.com/shadowdom');
  await browser.pause(3000)
  const innerEl = await $('#guid-generator').shadow$('#editField');
  await innerEl.click()
  await innerEl.setValue("zjcfjkasnkjfnalksjnl")
  await browser.pause(3000)
})

```

Mock:

```

describe('Mock Request Example and other', () => {
  it('should mock a request and send us joke', async () => {
    await browser.url('https://jokeapi.dev/');

    const mock = await browser.mock('https://v2.jokeapi.dev/joke/Any', {method: 'GET'})

    mock.respond({
      "error": false,
      "category": "Programming",
      "type": "single",
      "joke": "dfsdfsdfsdf",
      "flags": {
        "nsfw": false,
        "religious": false,
        "political": false,
        "racist": false,
        "sexist": false,
        "explicit": false
      },
      "id": 51,
      "safe": true,
      "lang": "en"
    }, {
      statusCode: 200,
      headers: { 'Content-Type': 'application/json' }
    });

    const sendRequestButton = await $('//button[contains(text()),"Send Request"]')
    await sendRequestButton.scrollIntoView()
    await browser.pause(3000)
    await sendRequestButton.click()
  })
})

```

```

    await browser.pause(10000)

});
    //const someVariable = browser.mock(url, { method, headers, responseHeader, body });
    // somevariable now has the mock
    // then I have to say him after mocking what data/step he needs to do.

it('should mock a request to JSONPlaceholder', async () => {

    await browser.url('https://jsonplaceholder.typicode.com/');

    const mockRunButton = await browser.mock('https://jsonplaceholder.typicode.com/posts/1', {
        method: 'GET',
        response: {
            body: {
                "userId": 1,
                "id": 1,
                "title": "Shorkari chakri korile desher sheba kora jay. Onno job behuda",
                "completed": false
            },
            statusCode: 200,
            headers: { 'Content-Type': 'application/json' }
        }
    });

    const runButton = await $('#run-button')
    await runButton.scrollIntoView()
    await browser.pause(3000)
    await runButton.click()
    await browser.pause(10000)

});

```

```

it('should mock a request and send us joke', async () => {
  await browser.url('https://play1.automationcamp.ir/register.html');

  const mock = await browser.mock('https://play1.automationcamp.ir/confirm',
    method: 'get'
  });

  const htmlResponse = `
    <html>
      <body>
        <h1>Mocked HTML Content</h1>
        <p>This is a mocked HTML response.</p>
      </body>
    </html>
  `;

  mock.respond(htmlResponse, {
    statusCode: 200,
    headers: { 'Content-Type': 'text/html' }
  });

  const firstName = await $('input[name="first_name"]');
  const lastname = await $('input[name="last_name"]');
  const email = await $('input[name="email"]');
  const password = await $('input[name="password"]');
  const confirmPassword = await $('input[name="confirm_password"]');
  const checkBox = await $('input[name="terms"]');
  const registerBtn = await $('#submit_button')

  await firstName.setValue("Noman")
  await lastname.setValue("Ibrahim")

```

```
await email.setValue("Ibrahim@mail.com")
await password.setValue("123456")
await confirmPassword.setValue("123456")
await checkBox.click()
await registerBtn.click()

await browser.pause(10000)
});
```

Class 11: Cache_Cookie_Debugger_Array

Class 12: Flash Message, Flash Drop-down

wait for:

If I do not know how much time to take a element to show then we can use these instead for using **browser.wait()**. **Because browser.wait() always makes the exact waiting.... If the elements found in that waiting time then it is also completes it waiting time but lower timeout method will proceed the next step as soon as it found the element.**

```
elem.waitForDisplayed({ timeout: 3000 })
elem.waitForClickable({ timeout: 3000 })
elem.waitForExist({ timeout: 3000 })
elem.waitForEnabled({ timeout: 3000 })
//https://play1.automationcamp.ir/expected_conditions.html
```

Class 13: Mocha, Calender

Calendar:

<https://primeng.org/calendar>

<https://www.globalsqa.com/demo-site/datepicker/#Icon%20Trigger>

//div[@class="ant-picker-calendar-date-value"][text()='14"]

<https://ant.design/components/calendar>

```
document.getElementById('inputField').value = "ksndlskndlks"  
span[@data-date="2024-6-4"]
```

```
it.only('Calendar ', async () => {  
  await browser.url('http://127.0.0.1:5500/claendar.html');  
  await browser.pause(3000)  
  const calendarInput = await $('#date-input')  
  await calendarInput.setValue("1/7/2024")  
  await browser.pause(3000)  
})
```

```
it('Calendar ', async () => {  
  await browser.url('http://127.0.0.1:5500/claendar.html');  
  await browser.pause(3000)  
  const calendarInput = await $('#date-input')  
  await browser.execute((input) => {input.value = '12/07/2023';}, calendarInput);  
  await browser.pause(3000)  
})
```

```
it.only('Calendar ', async () => {  
  await browser.url('https://primeng.org/calendar');  
  await browser.pause(1000)
```

```

const calendarInput = await $$("//span/input[@type='text']")
await calendarInput[0].scrollIntoView()
// await calendarInput[0].click()
const isClickable = await calendarInput[0].isClickable()
console.log("===== ",isClickable)
await calendarInput[0].click()
await browser.pause(1000)
await browser.execute((input) => {
    input.value = '01/2024';
}, calendarInput[0]);
await browser.pause(2000)
await browser.keys([Key.Enter])
await browser.pause(2000)

})

```

```

it.only('Calendar ', async () => {
    await browser.url('https://primeng.org/calendar');
    await browser.pause(1000)
    const calendarInput = await $$("//span/input[@type='text']")
    await calendarInput[13].scrollIntoView()
    // await calendarInput[0].click()
    const isClickable = await calendarInput[13].isClickable()
    console.log("===== ",isClickable)
    await calendarInput[13].click()
    await browser.pause(1000)
    await browser.execute((input) => {
        input.value = '01/2024';
    }, calendarInput[13]);
    await browser.pause(2000)
    await browser.keys([Key.Enter])
    await browser.pause(2000)

})

```

```

it.only('Calendar ', async () => {
    await browser.url('https://primeng.org/calendar');

```

```

    await browser.pause(1000)
    const calendarInput = await $$("//span/input[@type='text']")
    await calendarInput[17].scrollIntoView()
    // await calendarInput[0].click()
    const isClickable = await calendarInput[17].isClickable()
    console.log("===== ", isClickable)
    // await calendarInput[17].click()
    await browser.pause(1000)
    await browser.execute((input) => {
        input.value = '07/10/2024';
    }, calendarInput[17]);
    await browser.pause(2000)
    await browser.keys([Key.Enter])
    await browser.pause(2000)

})

it.only('Calendar ', async () => {
    await browser.url('https://www.globalsqa.com/demo-site/datepicker/');
    await browser.pause(1000)
    const iframe = await $("//iframe[contains(@data-src,'datepicker/default')]")
    await browser.switchToFrame(iframe)
    await browser.pause(1000)
    const dateFiled = await $("#datepicker")
    await browser.execute((input) => {
        input.value = '07/10/2024';
    }, dateFiled);
    await browser.pause(2000)
    // await browser.keys([Key.Enter])
    await browser.pause(2000)

})

it.only('Calendar', async () => {
    await browser.url('https://www.globalsqa.com/demo-site/datepicker/#DropDo
    await browser.pause(5000)

```

```

const iframe = await $("//iframe[contains(@data-src,'datepicker/dropdown')]")
await browser.switchToFrame(iframe)
await browser.pause(1000)
const dateFiled = await $("#datepicker")
await browser.execute((input) => {
  input.value = '07/10/2024';
}, dateFiled);
await browser.pause(2000)
// await browser.keys([Key.Enter])
await browser.pause(2000)
})

```

nth child chaining

```

it.only('Table', async function () {
  await browser.url('http://127.0.0.1:5500/table.html');
  await browser.pause(2000)
  const firstRow = await browser.$('#employeeTable tbody tr:nth-child(1)');
  const id = await firstRow.$('td:nth-child(1)').getText();
  console.log("===== ",id)
})

```